

¿Qué es un ciclo?

Un ciclo en python se considera un bucle finito (ya que debe tener un fin, sino, sería literalmente un código come memoria realizando el bucle infinitamente) que se detiene bajo una condición (dada por nosotros o por la extensión del recorrido del ciclo).

Pero, ¿cómo podríamos explicarlo de una forma más cercana a ustedes en sus primeros contactos con ciclos?

Imaginen que tienen 30 personas invitadas a sus cumpleaños y obviamente deben saludar a cada una de esas personas mediante código de python, hasta ahora, probablemente lo único que pueden imaginar sin ciclos es imprimir 30 veces "Hola, gracias por venir a mi cumpleaños!" pero con un ciclo bien implementado ustedes pueden generar ese mismo mensaje 30 veces sin necesidad de escribirlo una y otra y otra vez.

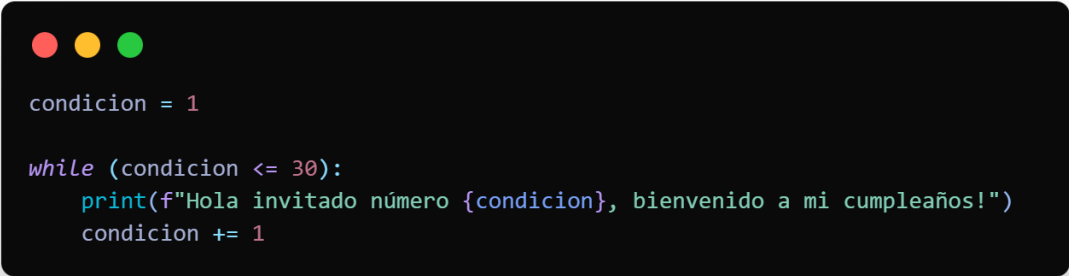
En resumen: Un ciclo nos sirve para reiterar un segmento de código hasta que se deje de cumplir una condición que nosotros le dimos (a eso me refiero a que se detiene bajo una condición, cuando deja de cumplirse, se detiene el ciclo).

Y como siempre, dejaré la estructura, pero como ahora tenemos 2 tipos de ciclos dividiré esto en.....

Ciclo while

Este es probablemente el ciclo que ustedes amarán y en algún punto van a querer utilizarlo siempre, lo cual no es lo ideal pero si pueden hacerlo y les funciona son completamente recibidos con sus códigos así mismo.

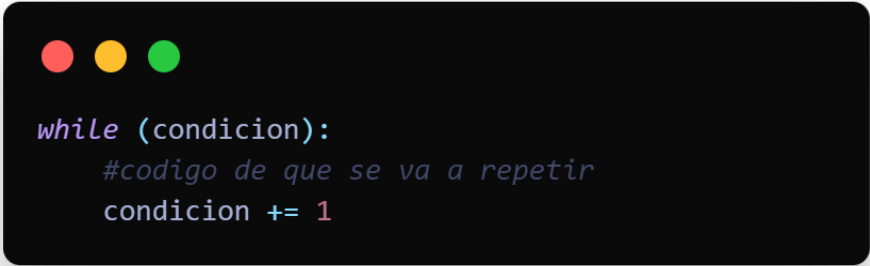
Su estructura se ve así:



```
condicion = 1

while (condicion <= 30):
    print(f"Hola invitado número {condicion}, bienvenido a mi cumpleaños!")
    condicion += 1
```

que más resumido aún se ve así:



```
while (condicion):  
    #codigo de que se va a repetir  
    condicion += 1
```

El ciclo while repetirá el código siempre que la condición que esté dentro se cumpla (la condición debe de venir de una variable externa al ciclo) pero para eso después del código SIEMPRE se debe afectar a la condición ya que sino tendríamos un bucle infinito y ya sabemos cómo termina eso. En el primer ejemplo de un ciclo while nuestra condición se inicializa en 1 y le decimos que **mientras** que la condición (1) sea menor o igual que 30, imprimirá un mensaje de saludo/bienvenida y, posteriormente, sumarle 1 a la condición para que se siga comparando, así repetidamente hasta llegar al número 30 (donde sigue siendo verdadera la condición dada, por ende, se repite el ciclo y la condición sube a 31, donde marca falsa la condición y da por terminado el bucle).

Pero, un error más que común es efectivamente no modificar la condición, provocando un bucle infinito. Otro error más que común sería no crear la variable externa que se vea afectada en el bucle while ya que el ciclo while no tendría de dónde sacar o modificar el dato que necesita analizar.

EJERCICIO PROPUESTO PARA VER SU ESTRUCTURA :0

Versión modificada de FizzBuzz:

Crear un programa que solicite un número entero, este número será ocupado dentro de un ciclo while.

Creando una variable, compararemos dentro del while esta variable hasta que llegue al número introducido.

En cada iteración (en cada vuelta del bucle), revisaremos los siguientes puntos:

- Si el número de la variable es divisible por 3 y por 5 imprimiremos "Fizzbuzz" dentro de esta vuelta

- Si el número es sólo divisible por 3, imprimiremos "Fizz" dentro de esta vuelta

- Si el número es sólo divisible por 5, imprimiremos "Buzz" dentro de esta vuelta

De no ser divisible por ninguno de estos números, directamente imprimiremos el número.



Ciclo For

Este ciclo lo odiarán, lo odiarán aquí y en C++, lo odiarán incluso si tienen que hacer algo parecido a un for dentro de algún código, da igual el lenguaje que ocupen... hasta que le tomen el ritmo.

Una vez le tomen el ritmo, amarán los ciclos for y intentarán meter uno si o si (incluso, analizarán la idea de utilizar un ciclo for en vez de un ciclo while muchas veces, si algún día les pasa, van por buen camino porque están pensando en más de 1 sola forma de abarcar un problema).

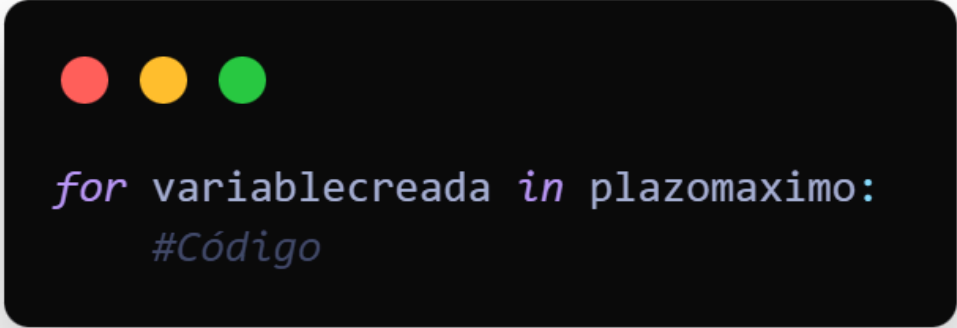
Primero, un ejemplo de su estructura con el ejercicio de cumpleaños:

```
for i in range(1,31):  
    print(f"Hola invitado número {i}, bienvenido a mi cumpleaños!")
```

Este código cumple exactamente la misma función que el código hecho con el ciclo while, pero, se abarcó de una distinta forma.

El ciclo for no necesita una variable externa, ya que “creamos” una variable dentro de los mismos parámetros del ciclo, lo que diferencia este ciclo del ciclo while es que un ciclo for necesita este formato de crear una variable y darle un plazo fijo de “vida”, por lo tanto, tenemos mayor control de lo que va a repetirse. El ciclo for no necesita modificar la variable creada dentro del código que escribamos, ya que esta misma va pasando por la siguiente iteración/vuelta del ciclo sin necesidad que le especifiquemos cómo lo queremos.

Su formato siempre será el mismo, por lo que sin código podría verse así:



```
for variablecreada in plazomaximo:  
    #Código
```

EJERCICIO PROPUESTO:

Crear una tabla de multiplicar desde el 1 al 10 con un número que se le pida al usuario!

Habiendo enseñado todo esto, que en verdad es bien corto de explicar y ver como materia como tal, les doy este tip que me solucionó la vida:

Siempre que sepan cuántas veces debe repetirse el ciclo, hagan sus bucles en base a ciclos for.

Siempre que sepan solamente una condición para detener los ciclos, hagan sus bucles en base a ciclos while.

Y siempre, pero siempre, **intenten traducir las palabras como código** (si/if, mientras/bucle, entrada/input, salida/print)